

Earley Parser Assignment Report

Ayush Kumar Gupta

2023114001

March 26, 2026

GitHub Repository: [Link](#)

Question 1: Earley Algorithm Chart

Task: Emulate the behaviour of the Earley algorithm on the sentence "*time flies like an arrow*" using the grammar rules provided in Figure 1.

Below is the final chart generated by the parser. (*Note: Log-probabilities are displayed by the parser's standard output, but the structural emulation ignores them for the standard Earley state transitions as requested*).

```
=====
EARLEY CHART
=====
```

Chart[0] (before reading any word)

```
[0,0] S -> • NP VP
[0,0] NP -> • N
[0,0] NP -> • Det N
[0,0] NP -> • N N
[0,0] N -> • time
[0,0] N -> • flies
[0,0] N -> • arrow
[0,0] Det -> • a
[0,0] Det -> • an
```

Chart[1] (after reading 'time')

```
[0,1] N -> time •
[0,1] NP -> N •
[0,1] NP -> N • N
[0,1] S -> NP • VP
[1,1] N -> • time
[1,1] N -> • flies
[1,1] N -> • arrow
[1,1] VP -> • V NP
[1,1] VP -> • V ADVP
[1,1] V -> • flies
[1,1] V -> • like
```

Chart[2] (after reading 'flies')

```
[1,2] N -> flies •
[1,2] V -> flies •
[0,2] NP -> N N •
[1,2] VP -> V • NP
[1,2] VP -> V • ADVP
```

```

[0,2] S -> NP • VP
[2,2] NP -> • N
[2,2] NP -> • Det N
[2,2] NP -> • N N
[2,2] ADVP -> • ADV NP
[2,2] VP -> • V NP
[2,2] VP -> • V ADVP
[2,2] N -> • time
[2,2] N -> • flies
[2,2] N -> • arrow
[2,2] Det -> • a
[2,2] Det -> • an
[2,2] ADV -> • like
[2,2] V -> • flies
[2,2] V -> • like

Chart[3] (after reading 'like')
[2,3] ADV -> like •
[2,3] V -> like •
[2,3] ADVP -> ADV • NP
[2,3] VP -> V • NP
[2,3] VP -> V • ADVP
[3,3] NP -> • N
[3,3] NP -> • Det N
[3,3] NP -> • N N
[3,3] ADVP -> • ADV NP
[3,3] N -> • time
[3,3] N -> • flies
[3,3] N -> • arrow
[3,3] Det -> • a
[3,3] Det -> • an
[3,3] ADV -> • like

Chart[4] (after reading 'an')
[3,4] Det -> an •
[3,4] NP -> Det • N
[4,4] N -> • time
[4,4] N -> • flies
[4,4] N -> • arrow

Chart[5] (after reading 'arrow')
[4,5] N -> arrow •
[3,5] NP -> Det N •
[2,5] ADVP -> ADV NP •
[2,5] VP -> V NP •
[1,5] VP -> V ADVP •
[0,5] S -> NP VP •

```

Question 2: Compute Tree Probabilities

Task: Compute the probability of each tree emitted by the parser using the rule probabilities from Figure 1.

The sentence yields two distinct parse trees. The probability of each tree is the product of the probabilities of all rules used in its derivation.

Tree 1 (Noun Phrase "time flies"):

- **Derivation:** [time flies]_{NP} [like [an arrow]_{NP}]_{VP}
- **Calculation:** $P(S \rightarrow NP VP) \times P(NP \rightarrow N N) \times P(N \rightarrow \text{time}) \times P(N \rightarrow \text{flies}) \times P(VP \rightarrow V NP) \times P(V \rightarrow \text{like}) \times P(NP \rightarrow D N) \times P(D \rightarrow \text{an}) \times P(N \rightarrow \text{arrow})$

- **Values:** $1.0 \times 0.25 \times 0.4 \times 0.2 \times 0.6 \times 0.5 \times 0.4 \times 1.0 \times 0.4 = \mathbf{0.00096}$

Tree 2 (Verb Phrase "flies"):

- **Derivation:** $[\text{time}]_{\text{NP}} [\text{flies} [\text{like} [\text{an arrow}]_{\text{NP}}]_{\text{ADVP}}]_{\text{VP}}$
- **Calculation:** $P(S \rightarrow \text{NP VP}) \times P(\text{NP} \rightarrow \text{N}) \times P(\text{N} \rightarrow \text{time}) \times P(\text{VP} \rightarrow \text{V ADVP}) \times P(\text{V} \rightarrow \text{flies}) \times P(\text{ADVP} \rightarrow \text{ADV NP}) \times P(\text{ADV} \rightarrow \text{like}) \times P(\text{NP} \rightarrow \text{D N}) \times P(\text{D} \rightarrow \text{an}) \times P(\text{N} \rightarrow \text{arrow})$
- **Values:** $1.0 \times 0.35 \times 0.4 \times 0.4 \times 0.5 \times 1.0 \times 1.0 \times 0.4 \times 1.0 \times 0.4 = \mathbf{0.00448}$

Tree 2 is approximately 4.67 times more probable under the provided grammar.

Question 3: Grammar Design for Ambiguity

Task: Design a grammar to give analyses for "*the man shot the soldier with a gun*" and test the parser implementation.

Designed Grammar

To capture the Prepositional Phrase (PP) attachment ambiguity (whether the man shot using the gun, or the soldier possessed the gun), the following grammar was designed with equal probabilities (0.5) for structural branching:

1.0	S	NP VP
0.5	NP	DT N
0.5	NP	DT N PP
0.5	VP	V NP
0.5	VP	V NP PP
1.0	PP	P NP
0.5	DT	the
0.5	DT	a
0.25	N	man
0.25	N	soldier
0.25	N	gun
1.0	V	shot
1.0	P	with

Final Chart & Parse Trees

The parser successfully emitted two valid analyses with identical probabilities (0.000122), reflecting the ambiguity of the PP attachment.

===== PARSE TREES =====

Tree 1 (VP Attachment: The man used a gun)
Probability : 0.0001220711

```
(S
  (NP (DT the) (N man))
  (VP (V shot)
    (NP (DT the) (N soldier))
    (PP (P with) (NP (DT a) (N gun))))))
```

Tree 2 (NP Attachment: The soldier had a gun)
Probability : 0.0001220711

```
(S
  (NP (DT the) (N man))
  (VP (V shot)
    (NP (DT the) (N soldier))
    (PP (P with) (NP (DT a) (N gun))))))
```

Question 4: Implementation & Efficiency

Task: Explain briefly how the submitted code solved the problems of Correctness and Efficiency.

Correctness

Every chart entry `chart[j][(ridx, dot, origin)]` stores a dictionary containing two keys: `'w'` (the highest log-probability) and `'backs'` (a list of all backpointers).

Tracking best derivation: When a new item is inserted, its log-weight is stored. If the same item arrives again via a different derivation, the new log-weight is compared against the stored `'w'`. If it is higher, `'w'` is updated. This guarantees the parser maintains the Viterbi weight.

Reacting to new derivations: The new backpointer is appended to the `'backs'` list if it is not already present, regardless of the weight. This decouples Viterbi probability tracking from tree structure tracking, allowing the post-parse step to exhaustively enumerate all distinct parse trees.

Efficiency

The algorithm strictly ensures $O(n^2)$ space and $O(n^3)$ time complexities:

Space $O(n^2)$: The chart contains $n + 1$ columns. Each item key is a tuple `(rule_idx, dot, origin)`. For a fixed grammar G , `rule_idx` and `dot` are constants. The `origin` index ranges from 0 to j (at most $n + 1$ values per column). Thus, each column holds at most $O(|G| \cdot n)$ items, yielding a total space complexity of $O(n^2)$.

Time $O(n^3)$: The PREDICT and SCAN operations require $O(|G|)$ and $O(1)$ work respectively. The dominant operation is COMPLETE. For each complete item starting at `origin` i and ending at j , it scans `chart[i]` for items waiting for the completed non-terminal. Since `chart[i]` has at most $O(|G| \cdot n)$ items, and there are $O(n^2)$ possible (i, j) pairs, the total time complexity is bounded by $O(n^3)$.

$O(1)$ Push & Duplicate Check: Each column `chart[col]` is implemented as a Python dictionary (hash table). Therefore, checking `key not in chart[col]` and inserting new items runs in amortized $O(1)$ time. This constant-time check is vital; without it, items could be added to the agenda multiple times, triggering redundant COMPLETE loops and leading to exponential time blowup.